পর্ব ৫ · টেস্টিং · ক্লাস ২৪ / ৩৭

যা পরীক্ষা করা হয়নি, তা নেই

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.১ (ইউনিট টেস্টিং)

১০৩টি টেস্ট · ১,৩৫৩ লাইন · ২২টি টেস্ট ক্লাস · ১৬টি `setUpTestData`

TestCase · setUpTestData · subTest

শুরু করুন প্রশ্ন দিয়ে

এক ঘণ্টা বেশি লাগিয়ে টেস্ট লিখব কেন?

হাত তুলুন: কে কখনো টেস্ট লিখেছেন? কে চালিয়েছেন?

কারণ একটি বাগ দুবার আসে না — যদি একবার লিখে রাখেন "এটা কাজ করে"।

IMS-এ **১০৩টি টেস্ট** আছে — সবচেয়ে বেশি বিলিংয়ে (৩৬), তারপর core-এ (১৯)। আজ শিখব এই টেস্টগুলোর গঠন, আর কেন কিছু ফাইলে **শূন্য** টেস্ট আছে।

শিক্ষকের নোট: বলুন: "টেস্ট লেখা সময় নয়, সময় বাঁচানো।" কিন্তু প্রথম সপ্তাহে ব্যয়টা বেশি লাগে — এটা স্বীকার করুন, নাহলে ছাত্ররা বিশ্বাস করবে না।

ধাপ ১

১০৩টি টেস্ট, ১১টি অ্যাপ

প্রতি অ্যাপে একটি `tests.py` — মোট ১,৩৫৩ লাইন

অ্যাপ	টেস্ট	লাইন
billing	**৩৬**	৪৩৬
core	১৯	২৪৯
assessments	১১	১৫৫
workspaces	১০	১১৮
accounts	৯	১১৪
students	৫	৬৮
attendance	৫	৮৫
academics · lessons	৪ · ৪	৫২ · ৭৬
reports · audit	**০ · ০**	—

বিলিংয়ে সবচেয়ে বেশি টেস্ট কেন? টাকা ভুল হলে ফেরানো কঠিন — আর নিয়মগুলোই জটিল (মেয়াদ · সিট · ইনভয়েস · গেট)।

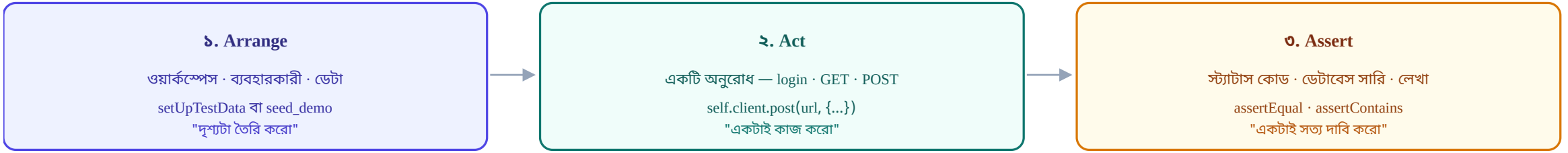
শূন্য টেস্ট = অন্ধ জায়গা: `reports/`-এর CSV আর `audit/`-এর পর্দা। কোড আছে, ভরসা নেই।

শিক্ষকের নোট: `python manage.py test apps` চালিয়ে ১০৩টি টেস্টের আউটপুট দেখান — সংখ্যাটা একবার চোখে দেখলে আর ভুল হয় না।

ধাপ ২

একটি টেস্টের গঠন — তিন ধাপ

Arrange · Act · Assert — নাম মনে রাখলেই যথেষ্ট



এক টেস্টে একটির বেশি Act নয় — নাহলে ব্যর্থ হলে বোঝা যায় না কোনটা ভেঙেছে

ভালো টেস্টের নাম পড়েই বোঝা যায় দৃশ্যটা: `test_expired_trial_goes_read_only_but_keeps_data` — তিনটি দাবি এক নামে।

শিক্ষকের নোট: বোর্ডে একটি খালি টেস্ট লিখুন তিন ভাগে ভাগ করে, তারপর কোডে পূরণ করুন। ছাত্ররা নিজের প্রজেক্টে একই ছক মানবে।

ধাপ ৩

`setUpTestData` বনাম `setUp`

১৬টি ক্লাস-স্তরের প্রস্তুতি · ২০টি মেথড-স্তরের

```
@classmethod
def setUpTestData(cls):
    call_command("seed_demo", verbosity=0)    # একবার, পুরো ক্লাসের জন্য

def setUp(self):
    self.client.force_login(self.teacher)     # প্রতিটি টেস্টের আগে

# পার্থক্য মনে রাখার নিয়ম
# setUpTestData → একবার চলে, তারপর প্রতিটি টেস্টের
#                 শেষে Django নিজেই রোলব্যাক করে
# setUp         → প্রতিটি টেস্টের আগে চলে (কোনো রোলব্যাক নেই)
```

প্রশ্ন	উত্তর
ডেটা কতবার তৈরি হয়?	`setUpTestData`-য় একবার
টেস্ট ডেটা বদলালে?	প্রতিটি টেস্ট শেষে রোলব্যাক — পরের টেস্ট পরিষ্কার
কেন `@classmethod`	ইনস্ট্যান্স নেই, তাই `cls` দিয়ে ডেটা রাখতে হয়
দুর্বলতা	`self` ব্যবহার করা যায় না — শেয়ারড অবজেক্ট বদলাবেন না

> বড় `seed_demo` একবার চালালে ৩৬টি বিলিং টেস্ট দ্রুত চলে — এটাই IMS-এর গতি-কৌশল।

শিক্ষকের নোট: লাইভে একটি টেস্টে ডেটা বদলে পরের টেস্ট চালান — দেখা যাবে বদল টেকেনি। রোলব্যাকের ধারণাটা এখানেই বাস্তব হয়ে ওঠে।

ধাপ ৪

`seed_demo` — ২৮১ লাইনের দৃশ্য

একটি প্রতিষ্ঠান, চার ভূমিকা, ছাত্র-ক্লাস-পরীক্ষা-হাজিরা

```
@classmethod
def setUpTestData(cls):
    call_command("seed_demo", verbosity=0)

SLUG = "sunrise-model-school"
PASSWORD = "DemoPass!2026"

LOGINS = [("admin@ims.local", "ORG_ADMIN"),
          ("coordinator@ims.local", "COORDINATOR"),
          ("teacher@ims.local", "TEACHER"),
          ("student@ims.local", "STUDENT")]

def _login(self, email):
    self.assertTrue(self.client.login(
        email=email, password=PASSWORD))
    self.client.get(reverse("workspaces:switch"),
        {"to": .pk})
```

কেন seed, কেন হাতে তৈরি নয়

হাতে ৩০টি সারি বানালে টেস্ট পড়া কঠিন হয়; seed আসল ডেটার মতোই বাস্তব।

idempotent

দুবার চালালেও ভাঙে না — তাই টেস্ট আর ডেমো দুই জায়গায় একই কমান্ড।

চার লগইন

একই ওয়ার্কস্পেসে চারটি ভূমিকা — তাই অনুমতির টেস্ট লেখা সহজ।

ঝুঁকি

seed বদলালে অনেক টেস্ট একসাথে ভাঙতে পারে — তাই slug ও পাসওয়ার্ড ধ্রুবক।

শিক্ষকের নোট: বলুন: "টেস্টের ডেটাও প্রোডাক্টের অংশ।" seed_demo ভাঙলে ডেমো আর টেস্ট দুটোই ভাঙে — তাই এটি যত্নে রাখা হয়।

ধাপ ৫

লগইনের তিন পদ্ধতি

`login()` ৩০ বার ব্যবহৃত · `force_login()` দ্রুত · `reverse()` ৩৪ বার

পদ্ধতি	কী করে	কখন
`self.client.login(email=…, password=…)`	সত্যিকারের লগইন প্রবাহ	লগইন নিজেই পরীক্ষার বিষয় হলে
`self.client.force_login(user)`	সেশন বানিয়ে দেয়	শুধু অনুমতি/পর্দা দেখতে চাইলে — দ্রুত
`self.client.get(reverse("…"))`	URL নাম থেকে পথ	সবসময় — হার্ডকোড করা URL নয়

`reverse()` ব্যবহার করলে URL বদলালে টেস্ট ভাঙে না — শুধু এক জায়গায় ঠিক করতে হয়।

`force_login` বেশি ব্যবহার করলে লগইন-সংক্রান্ত বাগ ধরা পড়ে না — তাই দুটোই দরকার।

শিক্ষকের নোট: প্রশ্ন: "টেস্টে `/w/sunrise-model-school/students/` হার্ডকোড করলে কী হয়?" উত্তর: slug বদলালে ২০টি টেস্ট ভাঙবে।

ধাপ ৬

কোন assert কখন — সংখ্যায় দেখুন

১২৯টি `assertEqual` · ১৭টি `assertContains` · ১২টি `assertRaises`

assert	কতবার	কী প্রমাণ করে
`assertEqual`	১২৯	মান সমান — স্ট্যাটাস, সংখ্যা, স্ট্রিং
`assertTrue` / `assertFalse`	২৯ / ৮	একটি শর্ত সত্য/মিথ্যা (`check_password`, `is_late`)
`assertContains`	১৭	পর্দায় লেখাটি সত্যিই দেখা যায়
`assertNotContains`	৫	যা দেখা যাবে না, তা সত্যিই দেখা যায় না
`assertRaises`	১২	ভুল ইনপুটে `ValidationError` / `PermissionDenied`
`assertRedirects`	**১**	— খুব কম; বাকিরা শুধু স্ট্যাটাস ৩০২ দেখে

`assertEqual(response.status_code, 302)` বলে না **কোথায়** গেল। `assertRedirects` একটাই ব্যবহার — বাকি জায়গায় গন্তব্য অপরীক্ষিত।

শিক্ষকের নোট: "৩০২ মানে কিছু একটা হয়েছে, কিন্তু কোথায়?" — এই ফাঁকটা দেখিয়ে ছাত্রদের নিজের টেস্টে `assertRedirects` ব্যবহার করতে বলুন।

ধাপ ৭

চার ভূমিকা, একটি টেস্ট — `subTest`

লুপের ভেতরে টেস্ট, কিন্তু ব্যর্থতা আলাদা করে রিপোর্ট হয়

```
def test_each_role_is_offered_only_its_own_links(self):
    for email, rules in EXPECTED_LINKS.items():
        with self.subTest(role=email):
            html = self._sidebar_html(email)
            for label in rules["must_show"]:
                self.assertIn(label, html)
            for label in rules["must_hide"]:
                self.assertNotIn(label, html)

# ৪ ভূমিকা × (দেখতে হবে + দেখাবে না) — এক টেস্টে
# ব্যর্থ হলে: "role='student@ims.local'" লেখা আসে
```

কেন subTest

লুপে সাধারণ assert দিলে প্রথম ব্যর্থতায় বাকি তিনটি ভূমিকা অপরীক্ষিত থেকে যায়।

রিপোর্ট

একই টেস্টে ৪টি আলাদা ফল — কোন ভূমিকা ভেঙেছে স্পষ্ট দেখা যায়।

কোথায় ব্যবহৃত

core-এ ৫ বার (ভূমিকা ও পর্দা), billing-এ ১ বার।

সীমা

এক টেস্টে ২০টি subTest থাকলে ব্যর্থতার তালিকা লম্বা হয় — ৪–৬ এর মধ্যে রাখুন।

শিক্ষকের নোট: ইচ্ছে করে একটি ভূমিকার EXPECTED_LINKS ভেঙে দেখান — আউটপুটে শুধু ওই ভূমিকাটি লাল হবে, বাকিরা সবুজ। এটাই subTest-এর পুরো মূল্য।

ধাপ ৮

৪০৩ বনাম ৪০৪ — টেস্টেই সিদ্ধান্ত

`test_non_member_gets_404_not_403` — কেন এটা জরুরি

```
def test_non_member_gets_404_not_403(self):
    other = User.objects.create_user(
        email="outsider@example.com", password="x", full_name="Out")
    self.client.force_login(other)
    resp = self.client.get(f"/w/{SLUG}/students/")
    self.assertEqual(resp.status_code, 404)

def test_teacher_cannot_open_administration_pages(self):
    self._login("teacher@ims.local")
    for name in ("audit list", "settings", "grading"):
        with self.subTest(page=name):
            resp = self.client.get(reverse(f"…:{name}", kwargs={...}))
            self.assertEqual(resp.status_code, 403)
```

পরিস্থিতি	সঠিক উত্তর
সদস্যই নন	**৪০৪** — প্রতিষ্ঠানটির অস্তিত্বই জানানো হয় না
সদস্য, কিন্তু ভূমিকা নয়	**৪০৩** — "আপনার অনুমতি নেই"
লগইন নেই	লগইনে রিডাইরেক্ট (৩০২)

ভুল হলে তথ্য ফাঁস: ৪০৩ বললে বোঝা যায় "এই ঠিকানায় একটা প্রতিষ্ঠান আছে"।

এই দুটি টেস্টই পর্ব ৪-এর ক্লাস ১৭-এর নিয়মগুলোকে লক করে রেখেছে — নিয়ম বদলালে টেস্ট লাল হবে।

শিক্ষকের নোট: প্রশ্ন করুন: "এক টেস্টে ৪০৩, আরেকটায় ৪০৪ — কেন আলাদা?" উত্তর: কারণ দুটো আলাদা ব্যবসায়িক সিদ্ধান্ত, আর টেস্ট হলো সেই সিদ্ধান্তের দলিল।

ধাপ ৯

টেস্ট চালানো — তিনটি কমান্ড

সব · এক অ্যাপ · এক টেস্ট

```
# সব টেস্ট
python manage.py test apps

# এক অ্যাপ
python manage.py test apps.billing

# একটি ক্লাস
python manage.py test apps.core.tests.RoleInterfaceTests

# একটি টেস্ট
python manage.py test
apps.billing.tests.ScopeTests.test_server_refuses_the_third_student

# দ্রুত পুনরাবৃত্তি (ডেটাবেস ফেলে দেয় না)
python manage.py test apps --keepdb -v 2

# আসল স্কুলের মতো স্মোক টেস্ট (রোল × URL ম্যাট্রিক্স)
python smoke_test.py
```

`--keepdb`

টেস্ট-ডেটাবেস তৈরি করতে ২–৫ সেকেন্ড বাঁচে; বারবার চালালে বড় পার্থক্য।

`-v 2`

প্রতিটি টেস্টের নাম দেখায় — কোনটি ধীর, তা চোখে পড়ে।

smoke_test.py আলাদা কেন

এটি `TestCase` নয় — আসল ডেমো ডেটাবেসে ৪টি লগইন দিয়ে **৬০টি URL** ঘোরে।

প্রতিদিনের অভ্যাস

কোড লেখার আগে নয়, লেখার পরে অন্তত একবার পুরো স্যুট — নাহলে ফাঁক জমে।

শিক্ষকের নোট: `--keepdb` ছাড়া ও সাথে দুইবার চালিয়ে সময়ের পার্থক্য দেখান — গতি নিজেই যুক্তি।

ধাপ ১০

টেস্টের নামই ডকুমেন্টেশন

ভালো নাম পড়লে কোড না দেখেই স্পেসিফিকেশন বোঝা যায়

নাম	যা বলে
`test_expired_trial_goes_read_only_but_keeps_data`	নিয়ম + সীমা + প্রতিশ্রুতি
`test_early_renewal_extends_instead_of_resetting`	ব্যবসায়িক সদিচ্ছা
`test_student_cannot_see_another_students_marksheet`	গোপনীয়তার সীমা
`test_no_role_sees_a_link_it_cannot_open`	দুই পক্ষ (মেনু ও সার্ভার) এক
`test_personal_workspace_is_never_gated`	ব্যতিক্রমটি স্পষ্ট
`test_domain_cannot_be_stolen_by_a_second_institution`	আক্রমণের নাম ধরে টেস্ট

ছক: `test_<অবস্থা>_<প্রত্যাশিত আচরণ>` — শর্ত আগে, ফলাফল পরে।

খারাপ নাম: `test_1`, `test_works`, `test_student` — ব্যর্থ হলে কিছুই বলে না।

শিক্ষকের নোট: `python manage.py test apps -v 2`-এর আউটপুট প্রিন্ট করান — পুরো তালিকাটাই এক পাতার স্পেসিফিকেশন।

সতর্কতা

পাঁচটি টেস্ট-বিরোধী অভ্যাস

প্রতিটির প্রতিকার এক লাইনে

অভ্যাস	পরিণতি ও প্রতিকার
১. এক টেস্টে অনেক assert	প্রথমটাই ফেল করলে বাকি দাবি অজানা → subTest বা ভাগ করুন
২. `setUpTestData`-র ডেটা বদলানো	পরের টেস্ট অপ্রত্যাশিত → শেয়ারড অবজেক্ট অপরিবর্তিত রাখুন
৩. শুধু স্ট্যাটাস ৩০২ দেখা	ভুল জায়গায় রিডাইরেক্টও পাশ → `assertRedirects`
৪. `force_login`-এ ভরসা	লগইনের বাগ চাপা পড়ে → অন্তত একটি সত্যিকারের `login()`
৫. সময়-নির্ভর টেস্ট	`timezone.localdate()` আসল তারিখ দেয় → `freeze_time` বা আপেক্ষিক তারিখ

পঞ্চমটি IMS-এ **এখনো কোথাও ব্যবহৃত হয়নি** — মেয়াদ-সংক্রান্ত টেস্টগুলো আপেক্ষিক তারিখ (`today ± timedelta`) দিয়ে লেখা, তাই আজ-কাল ভাঙে না।

শিক্ষকের নোট: পঞ্চমটার স্বাদ দিন: একটি টেস্টে `trial_ends_on = date(2026,11,5)` লিখে দেখান — আগামীকালই লাল হবে। "টেস্ট ক্যালেন্ডার পড়তে পারে না।"

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের প্রথম তিন টেস্ট

একটি মডেল, একটি ভিউ, একটি অনুমতি

১. মডেলের নিয়ম (৭ মিনিট)

একটি `clean()` বা property — সত্য/মিথ্যা দুই ক্ষেত্রেই assert।

২. ভিউয়ের উত্তর (৭ মিনিট)

`force_login` + `reverse` + স্ট্যাটাস ও `assertRedirects`।

৩. অনুমতি (৬ মিনিট)

ভুল ভূমিকায় ৪০৩, অ-সদস্যে ৪০৪ — দুটোই এক টেস্টে `subTest` দিয়ে।

- টেস্ট লেখা সময় নষ্ট নাকি সঞ্চয়?
- `setUpTestData` কেন `classmethod`?
- ৩০২ দেখলেই যথেষ্ট নয় কেন?
- seed ভাঙলে কী হয়?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: "একবারে ৩০টি টেস্ট লাল — আর তখন বোঝা যায় না কোড ভেঙেছে, নাকি seed।" তাই fixture-ও কোডের মতো যত্নের।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
এক টেস্ট, এক দাবি	১০৩ টেস্ট · ১,৩৫৩ লাইন · ২২ ক্লাস
প্রস্তুতি একবার, টেস্ট আলাদা	১৬ `setUpTestData` · `seed_demo` ২৮১ লাইন
URL নাম দিয়ে, হার্ডকোড নয়	`reverse()` ৩৪ বার
লুপে `subTest`	`EXPECTED_LINKS` — ৪ ভূমিকা, এক টেস্ট
৪০৩ বনাম ৪০৪ পরীক্ষিত	`test_non_member_gets_404_not_403`
যা পরীক্ষা হয়নি, তা অজানা	**reports ০ · audit ০**

শিক্ষকের নোট: শেষ সারিটা বারবার ফিরিয়ে আনুন — পরের ক্লাসেই এর বিস্তারিত: কোন ফাঁক কীভাবে বন্ধ করা যায়।

২৫

পরের ক্লাস — "কে কী দেখবে, প্রমাণসহ"

অনুমতি ও স্কোপের টেস্ট: `scoping.py` · ভূমিকা-ভিত্তিক ম্যাট্রিক্স · `smoke_test.py` · ক্রস-ওয়ার্কস্পেস ফাঁস

পর্ব ৫ · টেস্টিং · ক্লাস ২৫ / ৩৭

কে কী দেখবে, প্রমাণসহ

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.২ (অনুমতির টেস্টিং)
৩টি helper · ৮টি স্কোপিং টেস্ট · ৬০টি URL × ৪ ভূমিকা

visible_classes · 403 বনাম 404 · smoke matrix

শুরু করুন প্রশ্ন দিয়ে

"এই পাতা কি ও দেখতে পারবে?"

একটি প্রশ্ন নয়, দুটি — আর দুটির উত্তর দেয় দুটি আলাদা জিনিস

১. মেনুতে দেখায় কি? → `navigation.py`

২. সার্ভার খুলতে দেয় কি? → `scoping.py` + decorator

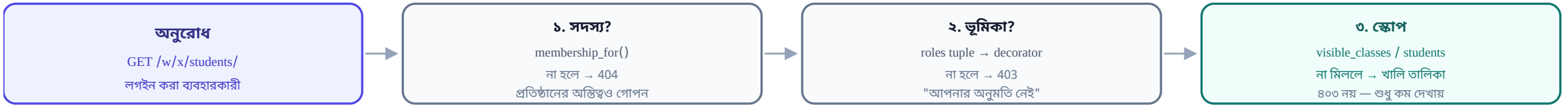
দুটো আলাদা জিনিস, কিন্তু **একই সত্য** বলা চাই। ক্লাস ২৪-এর টেস্ট `test_no_role_sees_a_link_it_cannot_open` ঠিক এই দুটোকে একসাথে বাঁধে — আজ দেখব ভেতরের কারিগরি।

শিক্ষকের নোট: বলুন: "মেনু লুকানো ভদ্রতা, অনুমতি নিরাপত্তা।" এই এক বাক্যেই পুরো ক্লাসের কাঠামো।

ধাপ ১

এক অনুরোধ, চারটি দেয়াল

প্রতিটি দেয়ালে আটকালে ফল আলাদা — ৩০২ · ৪০৪ · ৪০৩ · খালি তালিকা



লগইন নেই → 302 লগইন পর্দা (এটি decorator-এর প্রথম কাজ, তাই ছকে আলাদা দেখানো হয়নি)

ভুল ক্রম মানে ভুল উত্তর: সদস্য নয় বলে ৪০৩ দিলে প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়

স্কোপ আটকায় "কী দেখব", ৪০৩ আটকায় "কোন পাতায় ঢুকব" — তাই দুই ধরনের টেস্ট

এই চারটি দেয়ালই ক্লাস ১৭-এ লেখা হয়েছিল; আজকের কাজ সেগুলোকে **টেস্টে বেঁধে ফেলা** — যাতে ভবিষ্যতের পরিবর্তন নিয়ম ভাঙতে না পারে।

শিক্ষকের নোট: বোর্ডে চার দেয়াল লিখুন, তারপর প্রতিটির পাশে ফল লিখুন (302/404/403/খালি)। ছাত্ররা এই ছক মুখস্থ নয়, বুঝে নেবে।

ধাপ ২

`scoping.py` — একটি ফাইল, সব তালিকা

"একবার ঠিক করলে নয়টি ভিউয়ে ঠিক" — ফাইলের নিজের মন্তব্য

```python
def visible_classes(user, workspace, membership=None):
    membership = membership or membership_for(user, workspace)
    qs = ClassGroup.objects.filter(workspace=workspace,
                                   is_archived=False)
    if membership is None:
        return qs.none()

    if membership.role == Roles.ORG_ADMIN:
        return qs

    if membership.role == Roles.COORDINATOR:
        scopes = CoordinatorScope.objects.filter(
            workspace=workspace, user=user, is_active=True)
        dept_ids = list(scopes.exclude(department=None)
                        .values_list("department_id", flat=True))
        class_ids = list(scopes.exclude(class_group=None)
                         .values_list("class_group_id", flat=True))
        return qs.filter(models_q(dept_ids, class_ids))

    if membership.role == Roles.TEACHER:
        return qs.filter(assignments__teacher=user,
                         assignments__is_active=True).distinct()

    if membership.role == Roles.STUDENT:
        student = getattr(user, "student_profile", None)
        if student is None:
            return qs.none()
        return qs.filter(enrollments__student=student,
                         enrollments__is_active=True).distinct()
    return qs.none()
```

সদস্য না হলে `none()`

খালি queryset — `None` নয়, কারণ কলকারী সবাই `.count()` ডাকে।

শেষে `none()`

অজানা ভূমিকা = কিছু নয়। ডিফল্ট সব দেখানো বিপজ্জনক।

`membership=None` প্যারামিটার

ভিউ ইতিমধ্যে মেম্বারশিপ জানে — দুবার কুয়েরি হয় না।

`distinct()`

`assignments` ও `enrollments` জয়েনে ডুপ্লিকেট সারি আসে — নাহলে এক ক্লাস দুইবার।

শিক্ষকের নোট: প্রশ্ন: "শেষের `return qs.none()` না লিখলে কী হতো?" উত্তর: ভবিষ্যতের নতুন ভূমিকা সব ডেটা দেখতে পেত — নীরব বিপর্যয়।

ধাপ ৩

আটটি টেস্ট, আটটি সংখ্যা

`core/tests.py → ScopingTests` — পর্দা নয়, queryset পরীক্ষা

টেস্ট	প্রত্যাশা	দেখায়
`test_admin_sees_every_class_and_student`	সব ক্লাস · সব ছাত্র	একমাত্র "সব দেখে" ভূমিকা
`test_teacher_sees_only_assigned_classes`	১টি ক্লাস — "Class 8"	দ্বিতীয় শিক্ষকের "Class 9"
`test_teacher_only_sees_students_of_their_own_classes`	সেট সমান	`set(pk)` তুলনা — ক্রম নয়
`test_teacher_with_no_assignment_sees_no_students`	০ · ০	নতুন সদস্য, শূন্য অনুমতি
`test_coordinator_sees_only_their_scope`	০ → তারপর ১	স্কোপ মুছে/যোগ করে
`test_student_sees_only_own_record`	১ — "Rafid Hasan"	নাম মিলিয়েও যাচাই
`test_inactive_membership_sees_nothing`	০ · ০	`is_active=False`
`test_student_only_gets_published_homework`	`all(is_published)`	প্রকাশ না হলে ফাঁস নয়

সংখ্যা (১, ০, ২) ব্যবহার করা হয়েছে ইচ্ছাকৃতভাবে — "কিছু একটা আছে" নয়, **ঠিক কত** তা দাবি করা হয়েছে।

শিক্ষকের নোট: জোর দিন: `assertEqual(count(), 1)` অনেক শক্তিশালী `assertTrue(count() > 0)`-এর চেয়ে। "সংখ্যা দিয়ে দাবি করুন।"

ধাপ ৪

কোঅর্ডিনেটরের স্কোপ — যোগ-বিয়োগে প্রমাণ

"স্কোপ নেই" অবস্থাটাই সবচেয়ে বিপজ্জনক, তাই সেটাই আগে পরীক্ষা

```
coord = self._user("coordinator@ims.local")
# seed দিয়েছে Science ডিপার্টমেন্ট (Class 8)

CoordinatorScope.objects.filter(workspace=self.ws,
                                user=coord).delete()
self.assertEqual(scoping.visible_classes(coord, self.ws).count(), 0,
                 "a coordinator without a scope must see nothing")

CoordinatorScope.objects.create(
    workspace=self.ws, user=coord,
    department=Department.objects.get(workspace=self.ws,
                                      name="Science"))
self.assertEqual([c.name for c in
                  scoping.visible_classes(coord, self.ws)], ["Class 8"])
```

খালি অবস্থা আগে

ভুল করে "সব দেখানো" বাগ ধরা পড়ে কেবল তখনই, যখন শূন্য প্রত্যাশা করা হয়।

যোগ করার পর

ডিপার্টমেন্ট দিলে ঠিক "Class 8" — নাম দিয়ে যাচাই, সংখ্যা দিয়ে নয়।

`models_q()`

ডিপার্টমেন্ট **অথবা** সরাসরি ক্লাস — দুটোই এক ফিল্টারে (`Q |=`)।

তৃতীয় বার্তা

`assertEqual(..., 0, "a coordinator without a scope…")` — ব্যর্থ হলে কারণ লেখা আসে।

শিক্ষকের নোট: শেষ কার্ডটি দেখান: assert-এ বার্তা দিলে ৬ মাস পর ব্যর্থতা পড়েও বোঝা যায় কী ভাঙল। "টেস্টকে ভবিষ্যতের নিজের কাছে চিঠি লিখুন।"

ধাপ ৫

ক্রস-ওয়ার্কস্পেস — দুই স্কুল, এক টেস্ট

সবচেয়ে বিপজ্জনক ফাঁস: অন্য প্রতিষ্ঠানের সারি দেখা

```
@classmethod
def setUpTestData(cls):
    cls.teacher = User.objects.create_user(email="t@example.com", ...)
    cls.outsider = User.objects.create_user(email="o@example.com", ...)
    cls.ws  = Workspace.objects.create(name="School A", owner=cls.teacher)
    cls.ws2 = Workspace.objects.create(name="School B", owner=cls.outsider)
    Membership.objects.create(user=cls.teacher, workspace=cls.ws, role=Roles.TEACHER)
    Membership.objects.create(user=cls.outsider, workspace=cls.ws2,
role=Roles.ORG_ADMIN)

def test_other_workspace_cannot_open_the_student(self):
    self.client.force_login(self.outsider)     # School B-র অ্যাডমিন
    resp = self.client.get(reverse("students:detail",
                kwargs={"slug": self.ws.slug, "pk": self.student.pk}))
    self.assertEqual(resp.status_code, 404)    # 403 নয়
```

কেন ৪০৪

School B-র অ্যাডমিন School A-র সদস্য নন — তাই School A-র অস্তিত্বও জানানো হয় না।

slug-ই প্রথম প্রাচীর

ভুল slug দিলে `@workspace_view`-এ ৪০৪ — ভেতরের ভিউ পর্যন্ত পৌঁছায়ই না।

আরও একটি

`test_teacher_without_assignment_sees_no_students` — `assertNotContains(resp, "Rafid")`।

ফাঁক

ক্রস-ওয়ার্কস্পেস টেস্ট কেবল students অ্যাপে; attendance · assessments-এ নেই।

শিক্ষকের নোট: প্রশ্ন করুন: "৪০৩ দিলে কী তথ্য ফাঁস হতো?" উত্তর: বোঝা যেত ওই আইডিতে একটি ছাত্র আছে — তাই ৪০৪।

ধাপ ৬

ডেটাবেসকে দিয়েও প্রমাণ

কিছু নিয়ম ভিউয়ে নয়, constraint-এ — সেগুলোও টেস্ট হয়

```
def test_second_active_enrolment_is_blocked_by_the_database(self):
    from django.db import IntegrityError, transaction
    Enrollment.objects.create(workspace=self.ws, student=self.student,
                              class_group=self.cls, is_active=True)
    with self.assertRaises(IntegrityError):
        with transaction.atomic():          # না থাকলে টেস্ট ভেঙে যায়
            Enrollment.objects.create(workspace=self.ws,
                                      student=self.student,
                                      class_group=self.cls,
                                      is_active=True)

def test_ending_an_enrolment_keeps_history(self):
    e.is_active = False; e.end_reason = "Transferred"; e.save()
    e.refresh_from_db()
    self.assertIsNotNone(e.ended_on)          # save() নিজে তারিখ বসায়
    self.assertEqual(Enrollment.objects.filter(student=...).count(), 1)
```

`assertRaises` + `atomic`

IntegrityError-এর পর ট্রানজ্যাকশন ভাঙা থাকে — `atomic` ছাড়া পরের কুয়েরি ব্যর্থ।

ইতিহাস ≠ মুছে ফেলা

ভর্তি বন্ধ মানে সারি থাকবে, `ended_on` বসবে — সারি সংখ্যা ১-ই থাকে।

`refresh_from_db()`

DB-তে কী জমা হলো তা দেখতে বাধ্যতামূলক — মেমরির অবজেক্ট সত্য নয়।

কেন দরকার

ভিউ বাইপাস করে শেল/ইমপোর্টে ডেটা ঢুকলেও constraint আটকাবে — টেস্ট সেই প্রতিশ্রুতির প্রমাণ।

শিক্ষকের নোট: `transaction.atomic()` ছাড়া চালিয়ে দেখান — এরর আসবে "An error occurred in the current transaction"। এটাই বাস্তব ব্যথা।

ধাপ ৭

`smoke_test.py` — ৬০টি URL × ৪ ভূমিকা

আসল ডেমো ডেটাবেসে চলে · `TestCase` নয়, স্ক্রিপ্ট

```
ALL   = {"ORG_ADMIN", "COORDINATOR", "TEACHER", "STUDENT"}
STAFF = {"ORG_ADMIN", "COORDINATOR", "TEACHER"}
OWNER = {"ORG_ADMIN"}

PAGES = [
    ("/",                                  ALL,   "dashboard redirect"),
    (f"/w/{s}/students/",                  STAFF, "student list"),
    (f"/w/{s}/students/{student.pk}/status/", {"ORG_ADMIN", "COORDINATOR"}, "change status"),
    (f"/w/{s}/academics/scopes/",          OWNER, "coordinator scope"),
    (f"/w/{s}/reports/export/results.csv?exam={exam.pk}", STAFF, "results csv"),
    (f"/w/{s}/renew/",                     OWNER, "subscription (owner only)"),
    ...
]

for url, allowed, note in PAGES:
    expect_ok = role in allowed
    acceptable = (200, 302) if expect_ok else (403,)
    response = client.get(url)
    if response.status_code not in acceptable:
        wrong.append((url, response.status_code, acceptable[0]))
```

প্রশ্নটা ভিন্ন

"পাতাটা কাজ করে?" নয় — "এই ভূমিকার জন্য এটি **অনুমোদিত** না **প্রত্যাখ্যাত**?"

তিনটি কোডই ব্যর্থতা

৫০০, অথবা প্রত্যাশিত ৪০৩-এর জায়গায় ২০০ — দুটোই খারাপ।

আউটপুট

`STUDENT 57/60 as expected` — মিসম্যাচ থাকলে URL ও আশা-প্রাপ্তি ছাপে।

সীমা

কেবল স্ট্যাটাস দেখে — পর্দার ভেতরে অন্য স্কুলের নাম আছে কি না দেখে না।

শিক্ষকের নোট: `python smoke_test.py` চালিয়ে আউটপুট দেখান — ৪ ভূমিকার সারি। তারপর একটি সারি ইচ্ছে করে ভেঙে দেখান কীভাবে মিসম্যাচ দেখায়।

ধাপ ৮

ম্যাট্রিক্সের ব্যতিক্রম — দুটি লাইন

প্রতিটি ব্যতিক্রম একদিন বাগের উৎস; তাই স্পষ্ট করে লেখা

```
if url in ("/", f"/w/{s}/results/"):
    acceptable = (200, 302)

# কেন?
#  "/"          → সবার জন্য, শুধু গন্তব্য ভিন্ন
#  "/results/" → staff: exam list · student: নিজের ফল
#                 দুই ভূমিকাতেই ২০০ বা ৩০২ বৈধ

# বাকি সব ক্ষেত্রে:
acceptable = (200, 302) if expect_ok else (403,)
```

ব্যতিক্রম লিখতে হয়

নাহলে ভুল ব্যর্থতা (false alarm) আসে, আর মানুষ টেস্ট উপেক্ষা করতে শেখে।

তবু সীমা

প্রতিটি ব্যতিক্রমের পাশে কারণ লেখা — কোডই ডকুমেন্টেশন।

বিকল্প ডিজাইন

পাতাগুলো আলাদা করলে (`/results/` staff ও student-এ ভিন্ন URL) ব্যতিক্রম লাগত না।

শিক্ষকের নোট: "সব টেস্ট সবুজ রাখতে ব্যতিক্রম বাড়াবেন না" — প্রতিটি ব্যতিক্রমের দাম একদিনের বাগ-সন্ধান। ভালো ডিজাইন ব্যতিক্রম কমায়।

ধাপ ৯

মেনুর দিকটাও পরীক্ষা

`test_each_role_is_offered_only_its_own_links` — HTML পড়ে, স্ট্যাটাস নয়

```
EXPECTED_LINKS = {
  "teacher@ims.local": {
    "must_show": ["Students", "Attendance", "Homework",
                  "Materials", "Exams & results"],
    "must_hide": ["Audit log", "Workspace settings", "Grading",
                  "Coordinator scope", "My oversight area",
                  "My homework", "My results"],
  },
  "student@ims.local": {
    "must_show": ["My homework", "My results", "Materials"],
    "must_hide": ["Students", "Attendance", "Syllabus",
                  "Lesson log", "Exams & results", "Reports", ...],
  },
}

html = self._sidebar_html(email)
self.assertIn(label, html)        # must_show
self.assertNotIn(label, html)     # must_hide
```

দুই তালিকা কেন

`must_show` ছাড়া মেনু ফাঁকা হয়ে গেলেও টেস্ট পাশ করবে।

HTML-এ লেখা খোঁজা

সাইডবারের HTML স্ট্রিং নিয়ে `assertIn` — DOM পার্সিং নয়, তাই দ্রুত।

দুর্বলতা

লেবেল লেখা বদলালে (অনুবাদ!) টেস্ট ভাঙবে — তাই লেবেল স্থির রাখা হয়।

জোড়া

এটি + `smoke_test` = মেনু ও সার্ভার দুই দিকেই প্রমাণ।

শিক্ষকের নোট: "লেবেল যদি বাংলায় অনুবাদ করতেন?" — টেস্ট ভাঙবে। সমাধান: লেবেলের বদলে `url_name` বা HTML-এ `data-nav` অ্যাট্রিবিউট খোঁজা।

ধাপ ১০

ম্যাট্রিক্সটি এক নজরে

✓ = ২০০/৩০২ · ✗ = ৪০৩ — এই ছকটাই `smoke_test`-এর সারমর্ম

পথ	ADMIN	COORD	TEACHER	STUDENT
`/w/x/students/`	✓	✓	✓	✗
`/w/x/students/<pk>/status/`	✓	✓	✗	✗
`/w/x/academics/scopes/`	✓	✗	✗	✗
`/w/x/attendance/`	✓	✓	✓	✗
`/w/x/reports/`	✓	✓	✓	✗
`/w/x/materials/`	✓	✓	✓	✓
`/w/x/renew/`	✓	✗	✗	✗
`/platform/`	✓ (superuser)	✗	✗	✗

কলাম দেখুন, সারি নয় — শিক্ষকের কলামে একটাই ✗ বেশি (status), শিক্ষার্থীর কলামে প্রায় সব।

ম্যাট্রিক্সে ফাঁকা ঘর থাকলে সেটাই প্রশ্ন: "এই ভূমিকা এটি কেন দেখবে/দেখবে না?" উত্তর না থাকলে নকশাই অসম্পূর্ণ।

শিক্ষকের নোট: নতুন ফিচার যোগ করলে এই ছকে একটি সারি যোগ করতে বলুন — তারপর ভাবতে হবে কে দেখবে। "ছকটাই ডিজাইনের যন্ত্র।"

সতর্কতা

অনুমতির টেস্টে পাঁচটি ফাঁক

IMS-এ কোনটি আছে, কোনটি নেই — সৎ হিসাব

ফাঁক	অবস্থা ও প্রতিকার
১. শুধু helper পরীক্ষা, HTTP নয়	`ScopingTests` সরাসরি ফাংশন ডাকে → ভিউ ভুল helper ব্যবহার করলে ধরা পড়বে না
২. পর্দার ভেতরের বিষয়বস্তু অপরীক্ষিত	`assertNotContains` মাত্র ৫ বার → এক স্কুলের নাম অন্য পর্দায় থাকতে পারে
৩. ক্রস-ওয়ার্কস্পেস টেস্ট কম	students-এ আছে; attendance · assessments · lessons-এ নেই
৪. রিপোর্ট ও অডিটে শূন্য টেস্ট	CSV-তে স্কোপ ভুল হলে কেউ ধরবে না
৫. অ্যাডমিন = সব	ORG_ADMIN-এর কোনো সীমা নেই — ভুল ডিজাইন নয়, কিন্তু টেস্টে সীমাটা লেখা থাকা দরকার

প্রথমটি সবচেয়ে ধূর্ত: helper ঠিক, কিন্তু ভিউ অন্য helper ডাকছে — সব টেস্ট সবুজ, তবু ফাঁস।

শিক্ষকের নোট: প্রথম ফাঁকটা লাইভে দেখান: একটি ভিউয়ে helper বদলে দিন — কেউ ধরে না। তারপর বলুন: "তাই HTTP-স্তরের টেস্টও দরকার।"

অনুশীলন ও আলোচনা

২০ মিনিট — নিজের স্কোপ লিখুন ও ভাঙুন

তিনটি কাজ, তারপর চারটি প্রশ্ন

১. helper (৭ মিনিট)

`visible_<entity>(user, workspace)` — চার ভূমিকার শাখা, শেষে `none()`।

২. চারটি টেস্ট (৮ মিনিট)

admin = সব · teacher = ১ · student = নিজে · সদস্য নয় = ০।

৩. ইচ্ছাকৃত ফাঁস (৫ মিনিট)

helper থেকে `none()` সরিয়ে দেখুন কোন টেস্ট লাল হয় — আর কোনগুলো হয় না।

- মেনু লুকানো কেন যথেষ্ট নয়?
- ক্রস-ওয়ার্কস্পেসে ৪০৩ নয়, ৪০৪ কেন?
- `count()==1` বনাম `count()>0` — কোনটা ভালো?
- helper ঠিক, ভিউ ভুল — কীভাবে ধরবেন?

শিক্ষকের নোট: তৃতীয় কাজটাই আসল শিক্ষা — কিছু টেস্ট সবুজ থাকবে, কারণ তারা ওই শাখা ছোঁয়ই না। "কভারেজ মানে সংখ্যা নয়, শাখা।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
স্কোপ এক জায়গায়	`scoping.py` — `visible_classes` · `visible_students` · `visible_homework`
চার দেয়াল, চার উত্তর	302 · 404 · 403 · খালি তালিকা
সংখ্যা দিয়ে দাবি	১ · ০ · ২ — "কিছু আছে" নয়
ডেটাবেসের নিয়মও টেস্ট হয়	`assertRaises(IntegrityError)` + `atomic`
ভূমিকা × URL ম্যাট্রিক্স	`smoke_test.py` — ৪ লগইন, ৬০ পথ
helper টেস্ট ≠ HTTP টেস্ট	ফাঁক ১ — ভিউ অন্য helper ডাকলে ধরা পড়ে না

শিক্ষকের নোট: শেষ সারিটা পরের ক্লাসের সূচনা: কভারেজ সংখ্যা যা দেখায়, আর যা লুকায়।

২৬

পরের ক্লাস — "সংখ্যাটা মিথ্যা বলে না, কিন্তু সব বলে না"

কভারেজ ও যা বাকি: `coverage.py` · শাখা বনাম লাইন · ০ টেস্টের অ্যাপ · ফ্লেকি টেস্ট · টেস্ট পিরামিড

পর্ব ৫ · টেস্টিং · ক্লাস ২৬ / ৩৭

সংখ্যাটা মিথ্যা বলে না — সবও বলে না

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.৩ (কভারেজ ও ফাঁক)

১০৩ টেস্ট · ৬৬ HTTP · ৩৭ বিশুদ্ধ · ৫,৪৮০ লাইন অ্যাপ কোড

line vs branch · flaky · CI

শুরু করুন প্রশ্ন দিয়ে

"কভারেজ ৯২%" — এর মানে কী?

হাত তুলুন: কোডের ৯২% কাজ করে? ৯২% বাগ ধরা পড়বে?

কোনোটাই নয় — এর মানে শুধু "৯২% লাইন অন্তত একবার চলেছে"।

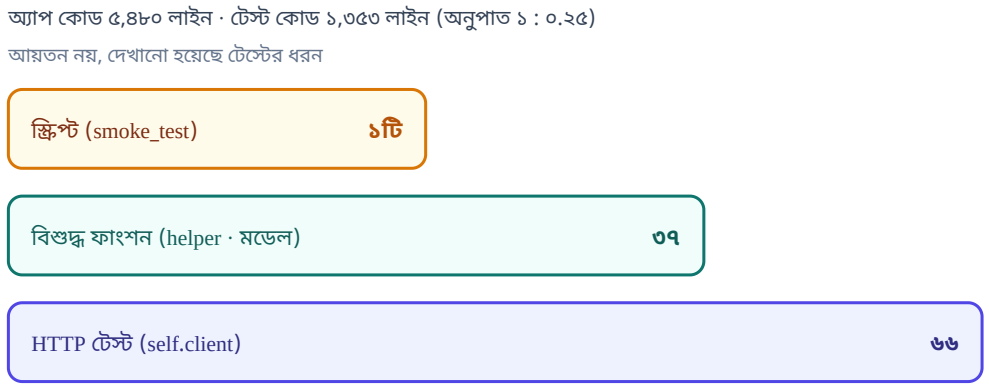
একটি লাইন চলা আর তার **সব শাখা** চলা এক নয়। তাই আজ: কভারেজ কী মাপে, কী মাপে না, আর IMS-এর ১০৩ টেস্ট কোথায় পাহারা দেয় — কোথায় দেয় না।

শিক্ষকের নোট: বলুন: "কভারেজ হলো স্পিডোমিটার, ব্রেক নয়।" বেশি সংখ্যা ভালো, কিন্তু নিরাপত্তার প্রমাণ নয়।

ধাপ ১

আমাদের স্যুটের আসল আকার

১০৩ টেস্ট · ১,৩৫৩ লাইন · অ্যাপ কোড ৫,৪৮০ লাইন



অ্যাপ	টেস্ট	HTTP
billing	৩৬	২৩
core	১৯	১০
assessments	১১	৬
workspaces	১০	৮
accounts	৯	৯
students · attendance	৫ · ৫	৩ · ৩
academics · lessons	৪ · ৪	১ · ৩
reports · audit	**০ · ০**	০

প্রতি ৪ লাইন কোডে ১ লাইন টেস্ট — শিল্পে ১:০.৫–১ সাধারণ; IMS নিচু দিকে, কিন্তু **সঠিক জায়গায়** ঘন।

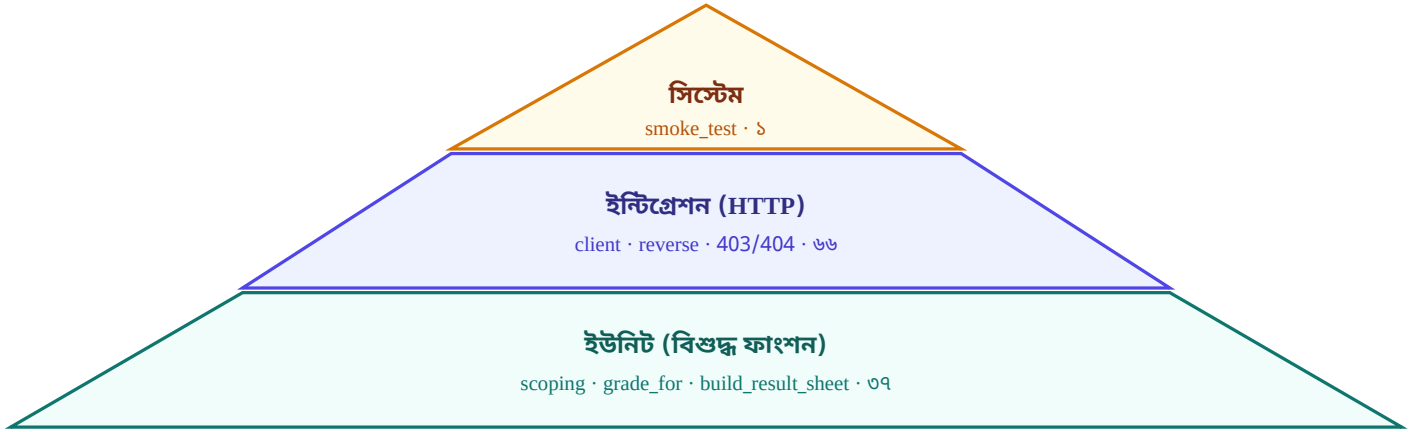
শিক্ষকের নোট: "HTTP ৬৬, বিশুদ্ধ ৩৭" — এই ভাগটাই টেস্ট পিরামিডের বাস্তব রূপ; পরের স্লাইডে কেন এটাই সঠিক।

ধাপ ২

পিরামিড — নিচে বেশি, উপরে কম

বিশুদ্ধ টেস্ট দ্রুত ও স্পষ্ট; HTTP টেস্ট ধীর কিন্তু বাস্তব

উপরের স্তর:
ধীর, ভঙ্গুর
কিন্তু আসল



নিচের স্তর:
দ্রুত, নির্দিষ্ট
কিন্তু "অ্যাপ" দেখে না

IMS-এ ইন্টিগ্রেশন বেশি — কারণ বেশিরভাগ বাগ ছিল **অনুমতি ও URL** নিয়ে, যা শুধু HTTP-তে ধরা পড়ে।

উল্টো পিরামিড (সব এন্ড-টু-এন্ড) ধীর আর ভঙ্গুর — ১০০০ টেস্ট ২০ মিনিট নিলে কেউ চালায় না।

শিক্ষকের নোট: বলুন: "পিরামিড নিয়ম নয়, অনুপাত।" অ্যাপের ধরন অনুযায়ী ভাগ বদলায় — IMS-এ অনুমতি বেশি, তাই ইন্টিগ্রেশন বেশি।

ধাপ ৩

কভারেজ মাপা — তিনটি কমান্ড

`coverage.py` · শাখা-সহ · কোন লাইন বাদ

```
pip install coverage

# ১. টেস্ট চালিয়ে হিসাব নিন
coverage run --branch manage.py test apps
coverage report -m           # কোন লাইন বাদ, দেখায়
coverage html                # htmlcov/index.html

# ২. শাখা মোডেই চালান
# --branch না দিলে if-এর ভেতরে না ঢুকলেও লাইন
# "covering" দেখায় — সবচেয়ে বড় মিথ্যা

# ৩. বাদ দিন যা মাপার দরকার নেই
# .coveragerc
[run]
branch = True
omit = */migrations/*, */tests.py, manage.py

[report]
exclude_lines =
    pragma: no cover
    if TYPE_CHECKING:
```

IMS-এ এখন নেই

`requirements.txt`-এ `coverage` নেই, `.coveragerc` নেই — অর্থাৎ আজ পর্যন্ত কভারেজ **কখনো মাপা হয়নি**।

migrations বাদ কেন

অটো-জেনারেটেড কোড — কভারেজের সংখ্যা বাড়ায়, আত্মবিশ্বাস দেয় না।

লক্ষ্য সংখ্যা নয়

৮০% নিয়ম হিসেবে দিলে লোকেরা অর্থহীন টেস্ট লেখে; বরং **ঝুঁকিপূর্ণ ফাইল** ১০০% চাই।

কোন ফাইল ঝুঁকিপূর্ণ

`scoping.py` · `permissions.py` · `billing/services.py` · `assessments/models.py`

শিক্ষকের নোট: ক্লাসে `coverage run --branch` চালিয়ে দেখান কোন ফাইল লাল — বিশেষ করে `reports/views.py` ও `audit/views.py`।

ধাপ ৪

লাইন কভারেজের মিথ্যা

একটি ফাংশন, চারটি শাখা — একটিমাত্র টেস্টে ৭৫% "কভার"

```
def is_writable(self):
    if self.status in (PAST_DUE, SUSPENDED, CANCELLED):
        return False                          # শাখা ১
    if self.status == TRIAL:
        return self.trial_ends_on is None or self.trial_ends_on >= today   # ২
    if self.status == ACTIVE:
        return self.current_period_end is None or self.current_period_end >= today
# ৩
    return False                              # শাখা ৪

# একটি ACTIVE সাবস্ক্রিপশন নিয়ে একবার কল করলেই
# coverage বলে: "সব লাইন চলেছে" — কিন্তু
# শাখা ১, ২ ও ৪ কখনো ছোঁয়া হয়নি।
```

প্রশ্ন	উত্তর
লাইন কভারেজ	"এই লাইন একবার চলেছে"
শাখা কভারেজ	"`if`-এর দুই দিকই চলেছে"
শর্ত কভারেজ	"`or`-এর দুই অংশ আলাদা চলেছে"
IMS-এ কত শাখা	`is_writable` ৪ · `visible_classes` ৫ · `activate` ২

`or`-এর দ্বিতীয় অংশ কখনো পরীক্ষা না হলে লাইন-কভারেজ ১০০% দেখায়, অথচ একটা সীমা পুরোপুরি অজ্ঞাত।

শিক্ষকের নোট: `is_writable`-এর চারটি রিটার্ন বোর্ডে লিখুন, তারপর জিজ্ঞেস করুন: "billing-এর ৩৬ টেস্ট কি চারটিই ছোঁয়?" উত্তর: প্রায় — মেয়াদ, ACTIVE, PAST_DUE ও SUSPEND সবই আছে।

ধাপ ৫

শূন্য টেস্টের দুই অ্যাপ

`reports/` ৩০০+ লাইন · `audit/` — দুটোই অন্ধ

যা অপরীক্ষিত	ভুল হলে
CSV কলামের ক্রম	Excel-এ ডেটা সরে যায়, কেউ টের পায় না
`_csv_response`-এর BOM	বাংলা নাম ভেঙে যায়
এক্সপোর্টের স্কোপ	শিক্ষক পুরো প্রতিষ্ঠানের ডেটা নামিয়ে ফেলেন
অডিট ফিল্টার (`action` · `actor`)	ভুল রেকর্ড দেখিয়ে তদন্ত বিভ্রান্ত
paginator ৪০/পাতা	শেষ পাতা কেউ দেখেন না

প্রথম তিনটি টেস্ট যা আজই লেখা উচিত: **১.** ছাত্র CSV নামাতে পারে না (৪০৩) · **২.** CSV-র প্রথম বাইট `\ufeff` · **৩.** অন্য প্রতিষ্ঠানের ছাত্র CSV-তে নেই

এগুলো লিখতে ৩০ মিনিট; এগুলো ছাড়া ঝুঁকি **চিরকাল**।

শিক্ষকের নোট: এই স্লাইডটা সরাসরি অনুশীলনে রূপ দিন — কেউ প্রথম টেস্টটা ক্লাসেই লিখে ফেলুক। "আজ ৩০ মিনিট, নাহলে অনন্তকাল।"

ধাপ ৬

ফ্লেকি টেস্ট — মাঝে মাঝে লাল

সবচেয়ে ক্ষতিকর: দল টেস্টকে বিশ্বাস করা বন্ধ করে দেয়

কারণ	লক্ষণ	প্রতিকার
১. সময়	আজ সবুজ, কাল লাল	`today ± timedelta` ব্যবহার — IMS এই পথেই
২. র‍্যান্ডম ডেটা	কখনো ১টি, কখনো ২টি সারি	`random.seed(7)` — **IMS-এ আছে**
৩. টেস্টের ক্রম	একা চালালে পাশ, সব একসাথে ফেল	প্রতিটি টেস্ট নিজের ডেটা বানাবে; শেয়ারড অবজেক্ট নয়
৪. ধীর I/O	মেইল/ফাইল টাইমআউট	`locmem` ইমেইল ব্যাকএন্ড · অস্থায়ী `MEDIA_ROOT`
৫. অনির্দিষ্ট dict ক্রম	সারির ক্রম বদলায়	`order_by()` সবসময়; `set()` দিয়ে তুলনা

`seed_demo`-তে `random.seed(7)` — একই ডেমো প্রতিবার হুবহু একই। এটাই **reproducible fixture**।

কিন্তু `test_result_sheet_handles_absent_zero_and_boundaries` আসল তারিখের ওপর নির্ভর করে না — তাই ২০৩০ সালেও চলবে।

শিক্ষকের নোট: প্রশ্ন: "একটি ফ্লেকি টেস্ট পেলে কী করবেন — আবার চালাবেন?" উত্তর: না, কারণটা খুঁজুন। "এলোমেলো লাল মানে এলোমেলো ভরসা।"

ধাপ ৭

CI নেই — যা এখন হাতের কাজ

প্রতিটি কমিটে কেউ নিজে `manage.py test` চালান

```
# .github/workflows/test.yml — যা এখনো যোগ করা হয়নি
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: ims, POSTGRES_USER: ims, POSTGRES_PASSWORD: ims }
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.13" }
      - run: pip install -r requirements.txt
      - run: python manage.py test apps
      - run: python smoke_test.py
```

এখন কী আছে

শুধু `run.sh`, `build.sh`, `smoke_test.py` — সব হাতে চালানো।

ঝুঁকি

"ভুলে যাওয়া" একটা অদৃশ্য বাগ: যেদিন কেউ টেস্ট চালায়নি, সেদিনই ভাঙা কোড মেইন-এ ঢোকে।

আগে কী যোগ করবেন

১. টেস্ট · ২. `manage.py check --deploy` · ৩. কভারেজ রিপোর্ট আর্টিফ্যাক্ট হিসেবে।

লোকাল গার্ড

CI-র আগে `pre-commit` হুক দিয়ে ফরম্যাট ও টেস্ট — ৩০ সেকেন্ডের বাধা।

শিক্ষকের নোট: "CI মানে আপনার চেয়ে ধৈর্যশীল সহকর্মী।" লোকালে ভুলে যাওয়া স্বাভাবিক — তাই স্বয়ংক্রিয় করা দরকার।

ধাপ ৮

স্যুট দ্রুত রাখা — ৪ সেকেন্ডের রহস্য

ধীর স্যুট = কেউ চালায় না = স্যুট নেই

কৌশল	লাভ
`setUpTestData` (১৬ জায়গায়)	ডেটা একবার, টেস্ট অনেক
`seed_demo` একবার চালানো	৩০ সারির বদলে একটি কল
`--keepdb`	টেস্ট-ডেটাবেস পুনঃতৈরি নয়
SQLite ডেমো সেটিংস	`config/settings/sqlite.py` — পোস্টগ্রেসের চেয়ে দ্রুত
বাইরের কল নেই	ইমেইল `console` / `locmem`, কোনো HTTP ক্লায়েন্ট নয়

১০৩ টেস্ট কয়েক সেকেন্ডে — তাই **প্রতিটি ছোট পরিবর্তনের পরেই** চালানো সম্ভব।

ধীর টেস্টের আসল খরচ সময় নয় — **মনোযোগ**। ৫ মিনিটের অপেক্ষায় কেউ আর চালায় না।

শিক্ষকের নোট: লাইভে টাইম করুন: `time python manage.py test apps`। সংখ্যাটা বোর্ডে লিখুন — সেটাই স্যুটের স্বাস্থ্য।

ধাপ ৯

সীমানার সংখ্যাগুলোই বাগের বাড়ি

৩২.৯৯% · ৩৩ · ০ · পূর্ণমান · একদিন আগে/পরে

ক্ষেত্র	সীমানা	টেস্টে আছে?
পাশ/ফেল	`pass_marks` = ৩৩ → ৩২ · ৩৩ · ৩৪	✔ `test_result_sheet_handles_absent_zero_and_boundaries`
গ্রেড ব্যান্ড	`min_percent` ও `max_percent`-এর প্রান্ত	✔ `test_grade_band_lookup`
পূর্ণমান	১০০-এর বেশি নম্বর	✔ `test_mark_above_full_marks_is_rejected`
অনুপস্থিত	`is_absent=True` + নম্বর দিলে?	✔ `test_absent_student_cannot_have_a_mark`
সিট সীমা	allowed−1 · allowed · allowed+1	✔ `test_server_refuses_the_third_student`
ট্রায়াল মেয়াদ	আজই শেষ · কাল শেষ · গতকাল শেষ	'আজ/গতকাল' পরোক্ষে; মাঝেরটি স্পষ্ট নয়

নিয়ম: প্রতিটি শর্তের **ঠিক আগে, ঠিক উপরে, ঠিক পরে** — তিনটি মান। সমতুল্য ভেতরের মান (৩৩, ৫৫, ৮৮) একটাই যথেষ্ট।

শিক্ষকের নোট: ট্রায়াল মেয়াদের তিনটি তারিখ দিয়ে `is_writable` চালিয়ে দেখান — মাঝেরটি (আজই শেষ) সবচেয়ে বেশি ভুল হয়, কারণ `>=` বনাম `>`।

ধাপ ১০

ঝুঁকি দিয়ে অগ্রাধিকার

সব জায়গায় ১০০% চাইলে কিছুই পাবেন না — তাই ক্রম

অংশ	ভুলের প্রভাব	সম্ভাবনা	অগ্রাধিকার
`scoping.py` · `permissions.py`	ডেটা ফাঁস	মাঝারি	**১ — আজ**
`billing/services.py` · `Invoice`	টাকা ভুল	মাঝারি	**১ — আজ**
`assessments` হিসাব	ভুল ফল	উচ্চ	**২**
`reports` CSV	ভুল রিপোর্ট	মাঝারি	**২** — ০ টেস্ট
`audit`	তদন্ত বিভ্রান্ত	কম	৩
`admin.py` · টেমপ্লেট	সৌন্দর্য	উচ্চ	৪ — চোখেই ধরা পড়ে

দুই মাত্রা: **ক্ষতি কত বড় × কত ঘন ঘন বদলায়**। ঘন বদলানো কোডে টেস্ট বেশি দরকার।

`reports/`-এর অবস্থান ২ নম্বরে, অথচ টেস্ট ০ — এই ফাঁকটাই আজকের ক্লাসের প্রমাণ।

শিক্ষকের নোট: দলের সঙ্গে ছকটা পূরণ করান — সবার অগ্রাধিকার এক নয়, কিন্তু "কোনটা ১ নম্বর" নিয়ে দ্বিমত থাকলে সেটাই আলোচনার শুরু।

সতর্কতা

কভারেজ নিয়ে পাঁচটি ভুল

সংখ্যাকে লক্ষ্য বানালে টেস্ট অর্থহীন হয়ে যায়

ভুল	পরিণতি
১. শতাংশকে লক্ষ্য বানানো	assert ছাড়া টেস্ট লেখা শুরু হয় — লাইন চলে, দাবি নেই
২. `--branch` বাদ দেওয়া	if-এর ভেতরে না ঢুকেও "কভার" দেখায়
৩. মাইগ্রেশন/টেস্ট কোড গোনা	সংখ্যা ফুলে ওঠে, আত্মবিশ্বাস বাড়ে না
৪. ভিউ-কভারেজে সন্তুষ্ট থাকা	helper-এর শাখা অজ্ঞাত থেকে যায় (ক্লাস ২৫-এর ফাঁক ১)
৫. রিপোর্ট না রাখা	প্রতি সপ্তাহে সংখ্যা বাড়ছে না কমছে, জানা যায় না

সবচেয়ে বড় ভুল: **ঝুঁকি ও কভারেজ আলাদা করে ভাবা**। `scoping.py`-এর ১০০% > `admin.py`-এর ১০০%।

শিক্ষকের নোট: শেষ লাইনটা বোর্ডে লিখুন। "কভারেজকে প্রশ্ন করুন — কোন ফাইল, কোন শাখা, কোন ঝুঁকি।"

অনুশীলন ও আলোচনা

২০ মিনিট — ফাঁক বন্ধ করুন

মাপুন, তারপর একটি ফাঁক বন্ধ করুন

১. মাপুন (৬ মিনিট)

`coverage run --branch manage.py test apps` → `coverage report -m`।

২. সবচেয়ে লাল ফাইল (৬ মিনিট)

শাখা-কভারেজ সবচেয়ে কম কোন ফাইলে — সেটি ঝুঁকিপূর্ণ কি না বিচার করুন।

৩. একটি টেস্ট (৮ মিনিট)

`reports`-এর জন্য একটি: ছাত্র CSV-তে ৪০৩ পাওয়া উচিত।

- লাইন কভারেজ ১০০% তবু বাগ — কীভাবে?
- ফ্লেকি টেস্ট কীভাবে চেনেন?
- CI না থাকলে সবচেয়ে বড় ঝুঁকি কী?
- কোন ফাইলে ১০০% চাইবেন?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: অনুমতি, স্কোপ, বিলিং হিসাব, মূল্যায়ন — যেখানে ভুল মানে টাকা বা গোপনীয়তা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর অবস্থা

নীতি	IMS-এ
কভারেজ লক্ষ্য নয়, মাপকাঠি	আজ **কখনো মাপা হয়নি** — coverage ইনস্টল নেই
শাখা কভারেজই আসল	`is_writable` -এর ৪ শাখা · `visible_classes` -এর ৫
পিরামিড মানুন	৩৭ বিশুদ্ধ + ৬৬ HTTP + ১ সিস্টেম
ফ্লেকি টেস্টের উৎস সময় ও র‍্যান্ডম	`random.seed(7)` · আপেক্ষিক তারিখ
দ্রুত স্যুট = ব্যবহৃত স্যুট	`setUpTestData` ×১৬ · `seed_demo` একবার
ফাঁক লিখে রাখুন	**reports · audit ০ টেস্ট · CI নেই**

শিক্ষকের নোট: শেষ সারিটাই এই কোর্সের শিক্ষণপদ্ধতি: গর্বের তালিকার পাশে ফাঁকের তালিকা — দুইটাই সত্য।

২৭

পরের ক্লাস — "টেস্ট আগে, কোড পরে"

টেস্ট ডিজাইন: কী টেস্ট করবেন না · সীমানা ও সমতুল্য শ্রেণি · TDD চক্র · রিফ্যাক্টর নিরাপদে · পর্ব ৫ সমাপ্তি

পর্ব ৫ · টেস্টিং · ক্লাস ২৭ / ৩৭ · পর্বের শেষ ক্লাস

টেস্ট আগে, কোড পরে

সিলেবাস: প্র্যাকটিক্যাল ৯ · টপিক ৯.৪ (টেস্ট ডিজাইন ও TDD)
লাল-সবুজ-রিফ্যাক্টর · সমতুল্য শ্রেণি · ৭টি প্রশ্ন · প্রমাণ-খাতা

red · green · refactor

শুরু করুন প্রশ্ন দিয়ে

টেস্ট আগে লিখলে কী লাভ?

হাত তুলুন: কে কখনো আগে টেস্ট লিখেছেন?

লাভ একটাই, কিন্তু বিশাল: টেস্টটা ব্যর্থ হতে দেখা — তারপর কোড লেখা।

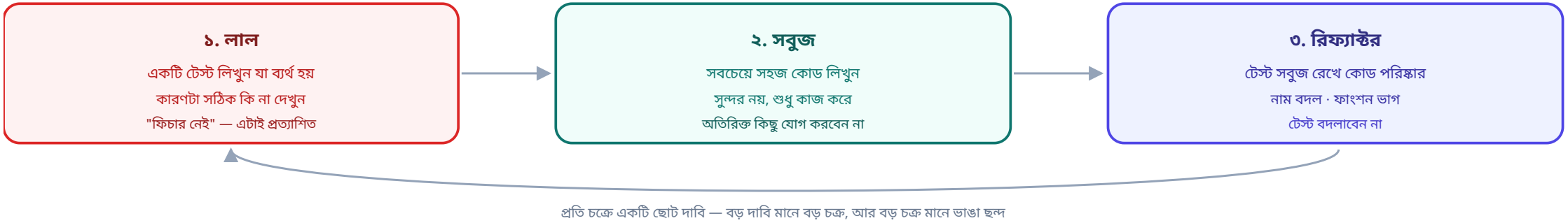
যে টেস্ট কখনো লাল হয়নি, সে টেস্ট আসলে কিছুই পরীক্ষা করছে কি না আপনি জানেন না। আজ: চক্রটি, কখন কোন টেস্ট, আর কী টেস্ট করবেন **না**।

শিক্ষকের নোট: বলুন: "সবুজ টেস্ট দেখে খুশি হবেন না — আগে লাল দেখুন।" এটাই TDD-র একমাত্র অলঙ্ঘনীয় নিয়ম।

ধাপ ১

তিন ধাপের চক্র

প্রতিটি চক্র ৫–১৫ মিনিট — বড় হলে ভেঙে ফেলুন



IMS-এ সব কোড TDD-তে লেখা হয়নি — কিন্তু যেখানে বাগ হয়েছিল (সেশন স্কোপ, গেটের URL), সেখানেই আগে টেস্ট লেখা হয়েছে।

শিক্ষকের নোট: সততার সাথে বলুন: "প্রথম থেকেই TDD কঠিন।" বাস্তব পথ: বাগ পেলেই আগে টেস্ট, তারপর ঠিক। এটাই সবচেয়ে বেশি কাজে দেয়।

ধাপ ২

একটি নতুন নিয়মে চক্রটি

"মেয়াদ শেষ হলে প্রতিষ্ঠান পড়তে পারবে, লিখতে পারবে না"

```
# ---- ১. লাল: আগে টেস্ট (is_writable এখনো নেই)
def test_expired_trial_goes_read_only_but_keeps_data(self):
    sub = self.sub
    sub.status = Subscription.Status.TRIAL
    sub.trial_ends_on = timezone.localdate() - timedelta(days=1)
    sub.save()
    self.assertFalse(sub.is_writable)                       # AttributeError = লাল
    self.assertEqual(AttendanceRecord.objects.filter(
        workspace=self.ws).count(), 42)                     # ডেটা অটুট

# ---- ২. সবুজ: সবচেয়ে ছোট কোড
@property
def is_writable(self):
    if self.status != self.Status.TRIAL:
        return False
    end = self.trial_ends_on
    return end is None or end >= timezone.localdate()

# ---- ৩. রিফ্যাক্টর: বাকি চার অবস্থা যোগ (নতুন টেস্টে, একই ফাইলে)
```

প্রথমে `AttributeError`

সেটাও এক ধরনের লাল — প্রমাণ করে টেস্টটি সত্যিই নতুন কোড ডাকছে।

"ডেটা অটুট" একই টেস্টে

নিয়মের সঙ্গে প্রতিশ্রুতিও পরীক্ষিত — নাহলে "read-only" মানে "মুছে ফেলা"ও হতে পারত।

সহজ কোড আগে

শুধু TRIAL হ্যান্ডল করে সবুজ করা — তারপর ACTIVE · PAST_DUE যোগ।

আসল কোডে

চূড়ান্ত সংস্করণে ৪টি শাখা — শেষে লেখা হয়েছিল, শুরুতে নয়।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে তিন ধাপ আলাদা রঙে লিখুন, প্রতিবার টেস্ট চালানোর ফল লিখুন। ছোট চক্রের শক্তি তখনই বোঝা যায়।

ধাপ ৩

সমতুল্য শ্রেণি — সব মান নয়, প্রতিটি দল

২,৫০০টি নম্বর পরীক্ষা নয়; প্রতিটি "ধরনের" একটি

ইনপুট	শ্রেণি	প্রতিনিধি মান
নম্বর (০–১০০)	নেগেটিভ · শূন্য · ভেতরে · পূর্ণমান · বেশি	−১ · ০ · ৫৫ · ১০০ · ১০১
সিট (allowed = ৩০০)	কম · ঠিক · বেশি	২৯৯ · ৩০০ · ৩০১
পাস নম্বর ৩৩	নিচে · সমান · উপরে	৩২ · ৩৩ · ৩৪
মেয়াদ	গতকাল · আজ · কাল	শেষ হয়েছে · আজই শেষ · আগামীকাল
টেক্সট	খালি · স্পেস · খুব লম্বা	`""` · `" "` · ৪০০ অক্ষর

IMS-এ এই ছক মেনে লেখা: `test_marks_entry_rejects_out_of_range_value` · `test_server_refuses_the_third_student`।

শূন্য ও খালি সবচেয়ে বেশি বাদ পড়ে — অথচ `ZeroDivisionError` ঠিক সেখানেই লুকিয়ে থাকে।

শিক্ষকের নোট: ছকটি বোর্ডে এঁকে ছাত্রদের নিজের ফিচারের জন্য পূরণ করতে বলুন — প্রতিটি দলের একটি মান। "শ্রেণি ভাবুন, সংখ্যা নয়।"

ধাপ ৪

নেগেটিভ টেস্ট আগে

"যা হবে না" প্রমাণ করা "যা হবে" প্রমাণের চেয়ে শক্ত

```
# BUILD_STATUS.md-এর ভাষায়
| Workspaces, memberships, roles, switching,
  isolation | negative tests: 404 on foreign slug |
| Result publication gate | student gets 403 before
  publish, 200 after |
| Teacher assignments | teacher without assignment
  sees nothing |

# কোডে সেই তিনটি
self.assertEqual(resp.status_code, 404)          # অন্য স্কুল
self.assertNotContains(resp, "Rafid")            # ছাত্র তালিকা ফাঁকা
self.assertTrue(all(h.is_published for h in mine))  # হোমওয়ার্ক
self.assertEqual(scoping.visible_classes(idle, ws).count(), 0)
```

কেন শক্ত

"কাজ করে" টেস্ট পাশ করে কম কোডেও; "কাজ করে না" টেস্ট ব্যর্থ হয় বাড়তি কোডে।

নিরাপত্তার একমাত্র ভাষা

ফাঁস প্রমাণ করতে হয় `assertNotContains` বা `count()==0` দিয়ে।

পুরোনো বাগ থেকে

প্রতিটি নেগেটিভ টেস্ট একদিনের বাস্তব ফাঁসের স্মৃতি — `test_no_role_sees_a_link_it_cannot_open`।

অনুপাত

১০৩ টেস্টে `assertRaises` ১২ · `assertNotContains` ৫ · `count()==0` বহুবার।

শিক্ষকের নোট: "নতুন ফিচারের জন্য প্রথম টেস্টটি লিখুন — 'কে এটা করতে পারবে না?'" এই এক লাইনেই বেশিরভাগ নিরাপত্তা-বাগ আটকায়।

ধাপ ৫

যা টেস্ট করবেন না

ভুল জায়গায় টেস্ট লিখলে ভঙ্গুর স্যুট, আর কম ভরসা

যা নয়	কারণ ও বিকল্প
Django-র নিজের কাজ	`ForeignKey` cascade, `reverse()` — ফ্রেমওয়ার্কের টেস্ট আছে
HTML-এর হুবহু গঠন	`<div class="…">` বদলালেই ভাঙে → লেখা বা স্ট্যাটাস দেখুন
প্রাইভেট মেথড	`_next_number` সরাসরি নয় — `Invoice.save()` দিয়ে প্রমাণ
তৃতীয় পক্ষের লাইব্রেরি	`reportlab`-এর ফন্ট, `Pillow`-এর রিসাইজ — শুধু "PDF তৈরি হয়" দেখুন
লগের হুবহু বাক্য	বানান বদলালেই ২০টি টেস্ট লাল → `action` কোড দেখুন
নিজের লেখা টেস্ট নকল	একই জিনিস তিন জায়গায় — একটি টেস্ট, একটি দাবি

নিয়ম: টেস্ট করুন **আপনার সিদ্ধান্ত**, ফ্রেমওয়ার্কের নয়। "কে অনুমতি দেবে" আপনার সিদ্ধান্ত; "`reverse()` কী ফেরে" নয়।

IMS-এ `test_each_role_is_offered_only_its_own_links` লেবেল লেখার ওপর নির্ভরশীল — ভঙ্গুরতার স্বীকারোক্তি।

শিক্ষকের নোট: প্রশ্ন করুন: "কেন লগের বাক্য পরীক্ষা করা খারাপ?" উত্তর: বাক্য নয়, ঘটনাটাই চুক্তি — বাক্য বদলানো উচিত কাজ, টেস্ট ভাঙা নয়।

ধাপ ৬

লাইভ ওয়ার্কথ্রু — `reports`-এর প্রথম টেস্ট

৬ ধাপ, ৮ মিনিট — ক্লাস ২৬-এর ফাঁক বন্ধ

```
# ১. ফাইল ও ক্লাস
# apps/reports/tests.py
class ExportScopeTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_demo", verbosity=0)
        cls.ws = Workspace.objects.get(slug="sunrise-model-school")
        slug = cls.ws.slug

    # ২. প্রথম টেস্ট: ছাত্র CSV নামাতে পারবে না
    def test_student_cannot_export_csv(self):
        self.assertTrue(self.client.login(
            email="student@ims.local", password="DemoPass!2026"))
        url = reverse("reports:export_students",
                      kwargs={"slug": slug})
        self.assertEqual(self.client.get(url).status_code, 403)

    # ৩. চালান → লাল? ৪. কোড ঠিক করুন → সবুজ
    # ৫. দ্বিতীয় টেস্ট: CSV-র প্রথম বাইট BOM
    # ৬. তৃতীয়: অন্য প্রতিষ্ঠানের নাম CSV-তে নেই
```

ধাপ ১–২ কেন আগে

`setUpTestData` + seed = ১০ সেকেন্ডে বাস্তব প্রতিষ্ঠান।

ফলাফল এখনই সবুজ

ভিউয়ে `*TEACHING` আছে — তাই ৪০৩ আসবে। প্রথম টেস্ট সবুজ হওয়া মানে নিয়ম আগেই ঠিক ছিল; এখন সেটি **লক** হলো।

BOM টেস্ট

`resp.content.startswith(b"\xef\xbb\xbf")` — এটাই আসল প্রমাণ।

সময়

তিনটি টেস্ট ~৩০ মিনিটের কাজ; ক্লাসে প্রথমটি একসাথে লিখুন।

শিক্ষকের নোট: সবসময় লাল দেখতে পাবেন না — পুরোনো নিয়ম লক করাও টেস্টের কাজ। "সবুজ মানেও ভালো, যদি দাবিটা অর্থবহ হয়।"

ধাপ ৭

টেস্ট-ডেটা: seed বনাম factory

IMS বেছে নিয়েছে seed — দুই দিকেরই দাম আছে

দিক	`seed_demo`	factory (যেমন factory_boy)
প্রস্তুতির গতি	১ লাইন	প্রতি টেস্টে কয়েক লাইন
পড়ার সহজতা	ডেটা লুকানো — `seed_demo` পড়তে হয়	টেস্টেই দেখা যায় কী বানানো হচ্ছে
সূক্ষ্ম নিয়ন্ত্রণ	কঠিন (সব কিছু আগেই আছে)	সহজ (ঠিক যা দরকার)
বাস্তবতা	উচ্চ — আসল অ্যাপের মতো	কম — শুধু প্রয়োজনীয় সারি
ভাঙার ঝুঁকি	seed বদলালে অনেক টেস্ট	ছোট, আলাদা

সেরা মিশ্রণ: **seed** দিয়ে বাস্তব জগৎ, **হাতে তৈরি** ছোট সারি দিয়ে সূক্ষ্ম নিয়ম (IMS-এ `EnrolmentTests` ঠিক এটাই করে)।

`seed_demo`-এর ভেতরের সংখ্যা (৪২টি অনুপস্থিতি, "Class 8") টেস্টে হার্ডকোড করা আছে — seed বদলালে ভাঙবে। বিকল্প: টেস্টে `seed` থেকে সংখ্যা পড়ে নেওয়া।

শিক্ষকের নোট: প্রশ্ন: "কেন seed-এর সংখ্যা হার্ডকোড করলে ঝুঁকি?" উত্তর: fixture আর টেস্টের মধ্যে অদৃশ্য চুক্তি তৈরি হয় — এক পাশ বদলালে অন্যপাশ নীরবে ভাঙে।

ধাপ ৮

রিফ্যাক্টর — টেস্ট হলো জাল

ক্লাস ১৮-এর দুই mixin একসাথে করার নিরাপদ পথ

```
# আগে: দুটি mixin, একই কাজ
# apps/academics/forms.py  → BootstrapFormMixin
# apps/billing/forms.py    → ImsFormMixin

# রিফ্যাক্টরের ক্রম (টেস্ট আগে!)
def test_forms_all_get_design_classes(self):     # ১. জাল পাতুন
    f = StudentForm(workspace=self.ws).fields
    self.assertIn("form-control", f["full_name"].widget.attrs["class"])
    self.assertIn("form-select", f["class_group"].widget.attrs["class"])

# ২. সবুজ দেখে নিন (এখনো দুই কপি)
# ৩. একটিকে apps/core/forms.py-তে সরান, অন্যটি ইমপোর্ট করে
# ৪. আবার পুরো স্যুট — ১০৩ টেস্ট সবুজ মানে আচরণ বদলায়নি
```

জাল আগে

রিফ্যাক্টরের আগে টেস্ট না থাকলে "কাজ করছে" প্রমাণ করার উপায় নেই।

আচরণ অপরিবর্তিত

রিফ্যাক্টরের একমাত্র অনুমোদন: **কোনো টেস্ট বদলানো যাবে না**।

এক ধাপে এক কাজ

নাম বদল · সরানো · ভাগ করা — আলাদা কমিট, আলাদা স্যুট রান।

সীমা

টেস্ট নেই এমন জায়গায় রিফ্যাক্টর = অনুমান। আগে টেস্ট, পরে সাহস।

শিক্ষকের নোট: "টেস্ট ছাড়া রিফ্যাক্টর মানে চোখ বন্ধ করে সিঁড়ি ভাঙা।" এটাই সবচেয়ে কাজে লাগা রূপক।

ধাপ ৯

প্রমাণ-খাতা — `BUILD_STATUS.md`

দাবির পাশে প্রমাণ; আর যা হয়নি, তাও লেখা

```
| Area | Evidence |
|---|---|
| Custom user model, email login | 43 tests + screenshots |
| Workspaces, roles, isolation | negative tests: 404 on foreign slug |
| Result publication gate | 403 before publish, 200 after |
| Marksheet + PDF | `%PDF` header asserted |

## Verified numbers
python manage.py test        ->  90 tests, 90 OK
python smoke_test.py         ->  55 pages x 4 roles
manage.py check --deploy     ->  no issues

## Remaining work (not faked, not skipped)
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite
```

"Evidence" কলাম

প্রতিটি দাবির পাশে কী প্রমাণ করে — টেস্টের সংখ্যা, নেগেটিভ টেস্ট, স্ক্রিনশট।

সংখ্যাগুলো চালিয়ে পাওয়া

"৯০ টেস্ট OK" — কমান্ডের আউটপুট, অনুমান নয়। আজ ১০৩ (স্লাইস বাড়ায়)।

"Remaining work"

যা বাকি তা **লেখা** — লুকানো নয়। এটাই পেশাদারির চিহ্ন।

কেন দরকার

নতুন ডেভেলপার (বা ভবিষ্যতের আপনি) ৫ মিনিটে অবস্থা বুঝে নিতে পারেন।

শিক্ষকের নোট: README-তে এখনো "13 tests" লেখা আছে (স্লাইস ১-এর স্ন্যাপশট) — ডকুমেন্টেশন পুরোনো হয়ে যাওয়ার বাস্তব উদাহরণ। "খাতা রাখলে তাজা রাখতে হবে।"

ধাপ ১০

নতুন ফিচারের সাতটি প্রশ্ন

প্রতিটি প্রশ্ন = অন্তত একটি টেস্ট

প্রশ্ন	টেস্টের ধরন
১. কে এটি খুলতে পারবে?	ভূমিকা × URL ম্যাট্রিক্স (৪০৩)
২. অন্য প্রতিষ্ঠান কী দেখবে?	ক্রস-ওয়ার্কস্পেস ৪০৪
৩. ভুল ইনপুটে কী হয়?	`assertRaises` বা `assertContains(..., "error")`
৪. খালি অবস্থায় কী দেখায়?	`{% empty %}` / `count()==0`
৫. মেয়াদ শেষে কী হয়?	`is_writable` · গেট
৬. দুইবার একই কাজ করলে?	ডুপ্লিকেট · `IntegrityError` · idempotent
৭. কে, কখন, কী করল?	`AuditLog` সারি আছে কি না

সাতটিই এক টেস্টে ঠেসে দেবেন না — সাতটি আলাদা টেস্ট, প্রতিটির নামে প্রশ্নটির উত্তর।

শিক্ষকের নোট: ছকটা প্রিন্ট করে দেওয়ার মতো — নতুন ফিচারের PR-এ এই সাতটি চেকবক্স রাখলে রিভিউ সহজ হয়।

ধাপ ১১

পুরো স্যুট একসাথে — এক আদেশে প্রমাণ

৫টি কমান্ড · ১০৩ টেস্ট · ৫১.৫ সেকেন্ড (SQLite)

```
# ১. পূর্ণ স্যুট (লোকাল, SQLite)
python manage.py test --settings=config.settings.sqlite
#   Ran 103 tests in 51.469s
#   OK

# ২. দ্রুত পুনরাবৃত্তি: ডিবি নতুন করে বানাবে না
python manage.py test --settings=config.settings.sqlite --keepdb

# ৩. সমান্তরাল (কোর অনুযায়ী প্রসেস)
python manage.py test --settings=config.settings.sqlite --parallel 4

# ৪. পাতা-স্তরের চোখ: ৫৫ পাতা × ৪ ভূমিকা
python smoke_test.py

# ৫. ডিপ্লয়-প্রস্তুতি: গোপনীয়তা, HSTS, ডিবাগ
python manage.py check --deploy
```

৫১.৫ সেকেন্ড মানে

প্রতি টেস্ট গড়ে ০.৫ সেকেন্ড — কফি খেতে যাওয়ার দরকার নেই, তাই কেউ ফাঁকি দেবে না।

--keepdb

টেস্ট-ডিবি মুছে নতুন করে মাইগ্রেট করে না — বারবার চালানোর জন্য।

--parallel

SQLite-এ লাভ কম, PostgreSQL-এ বেশি। প্রতিটি শ্রমিক আলাদা ডিবি পায়।

smoke_test আলাদা কেন

seeded ডেটায় **প্রতিটি URL × প্রতিটি ভূমিকা**; ৫০০ হলেই ব্যর্থ।

শিক্ষকের নোট: ক্লাসে ১ নম্বর কমান্ডটি চালিয়ে "Ran 103 tests ... OK" লাইনটি দেখান — এটাই কোর্সের সবচেয়ে বড় প্রমাণ।

ধাপ ১২

মিউটেশন মহড়া — নিজের স্যুটকে আক্রমণ

ইচ্ছাকৃত বাগ ঢুকিয়ে দেখুন টেস্ট ধরে কি না · আজ সত্যিই চালানো হয়েছে

#	ইচ্ছাকৃত পরিবর্তন	ফলাফল
A	`visible_homework` থেকে ছাত্রের `is_published=True` সরানো	**০ টেস্ট লাল** — স্যুট টের পায়নি
B	`visible_exams` থেকে একই ফিল্টার সরানো	**০ টেস্ট লাল**
C	শিক্ষকের `visible_classes` স্কোপ সরিয়ে সব ক্লাস দেওয়া	**৩ টেস্ট লাল** — `test_teacher_sees_only_assigned_classes` · `test_teacher_with_no_assignment_sees_no_students` · `EnrolmentTests.test_teacher_without_assignment_sees_no_students`

০ লাল মানে "বাগ নেই" নয় — "টেস্ট নেই"

A-এর কারণ: seed-এ একটিও অপ্রকাশিত হোমওয়ার্ক নেই, তাই `all(...)` খালি তালিকায় সত্য।

C কেন ধরা পড়ল

কারণ প্রতিটি স্কোপ নিয়মের জন্য **প্রতিপক্ষ** (অ্যাসাইনমেন্ট নেই এমন শিক্ষক) fixture-এ আছেন।

হাতের এই মহড়াই "মিউটেশন টেস্টিং"-এর হালকা রূপ; স্বয়ংক্রিয় করতে `mutmut` বা `cosmic-ray`, কিন্তু ৩টি মিউটেশন হাতে করলেই বোঝা যায় স্যুট কতটা শক্ত।

শিক্ষকের নোট: A আর B একসাথে দেখান — শিক্ষার্থীরা অবাক হবে যে ১০৩টি "OK" থাকা সত্ত্বেও দুটি নিরাপত্তা নিয়ম অরক্ষিত। এটাই এই ক্লাসের সবচেয়ে বড় শিক্ষা।

ধাপ ১৩

শূন্য-প্রমাণ টেস্ট — চার রূপ

টেস্ট সবুজ, কিন্তু কিছুই প্রমাণ করছে না

রূপ	উদাহরণ
১. প্রতিপক্ষই নেই	seed-এ অপ্রকাশিত হোমওয়ার্ক নেই → A/B টেস্ট টের পায় না
২. সব মান একই	সব নম্বর ৭০–৯৮ → সীমানা (−১, ০, ১০১) কেউ ছোঁয় না
৩. ব্যতিক্রম গিলে ফেলা	`try: … except: pass` — `assertRaises` কখনো ব্যর্থ হয় না
৪. মান যাচাই নেই	শুধু `status_code == 200`; পর্দায় কী আছে দেখা হয়নি

লক্ষণ: মিউটেশন চালালেই ধরা পড়ে — কোড ভাঙার পরও টেস্ট সবুজ।

```
# প্রতিকার: প্রতিপক্ষ নিজে বানান
def test_student_never_sees_unpublished_homework(self):
    hw = Homework.objects.filter(workspace=self.ws).first()
    hw.is_published = False          # শত্রু সারি
    hw.save()

    seen = {h.title for h in
            scoping.visible_homework(self.student, self.ws)}
    self.assertNotIn(hw.title, seen)   # এখন লাল হবে

# তারপর কোড ঠিক করুন → সবুজ
# এই একটি টেস্টই A-মিউটেশন ধরে ফেলবে
```

নিয়ম: **প্রতিটি নেগেটিভ টেস্টে একটি শত্রু সারি** নিজের হাতে তৈরি করুন — seed-এর ওপর ভরসা নয়।

শিক্ষকের নোট: বোর্ডে লিখুন: "সবুজ টেস্ট ≠ প্রমাণ। প্রমাণ = লাল হওয়ার ক্ষমতা।" এরপর ধাপ ১২-এর A মিউটেশনটি আবার চালিয়ে দেখান নতুন টেস্টটি লাল হয় কি না।

ধাপ ১৪

স্যুটের স্বাস্থ্য — ছয়টি সংকেত

সংখ্যা নয়, লক্ষণ দেখুন

সংকেত	খারাপ লক্ষণ	IMS-এ আজ
গতি	এক রানে ৫ মিনিটের বেশি → কেউ চালায় না	১০৩ টেস্ট · ৫১.৫ সেকেন্ড
নির্ধারিততা	প্রতিবার আলাদা ফল	`random.seed(7)` + `setUpTestData`
skip	“পরে ঠিক করব” জমে থাকা	০ skip
নাম	একই নাম দুই অ্যাপে → রিপোর্টে গুলিয়ে যায়	`test_teacher_without_assignment_sees_no_students` দুই অ্যাপে আছে
ব্যর্থতার বার্তা	শুধু `False is not true`	`"unpublished homework leaked to a student"`
নিরাপত্তা-আবরণ	এক স্তরে গার্ড, টেস্ট অন্য স্তরে	ভিউ-স্তরে পরীক্ষিত (৪০৩→২০০), হেল্পার-স্তরে নয়

শেষ সারিটি সবচেয়ে সূক্ষ্ম: **প্রতিরক্ষা দুই স্তরে, প্রমাণ এক স্তরে** — তাই দ্বিতীয় স্তর নীরবে সরে গেলেও স্যুট খুশি থাকে।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের স্যুটে এই ছয়টি সংকেত মিলিয়ে একটি স্কোর লিখতে — কোনটিতে সবচেয়ে দুর্বলতা, সেটাই পরের কাজ।

ধাপ ১৫

ক্লাসে ডেমো স্ক্রিপ্ট — ৮ মিনিট

টার্মিনালে যা চালাবেন, পর্দায় যা দেখাবেন

মিনিট	কমান্ড	পর্দায় যা দেখাবেন
০–১	`manage.py test … --settings=config.settings.sqlite`	শেষ দুই লাইন: `Ran 103 tests in 51.469s` · `OK`
১–৩	`--verbose 2` সহ একই কমান্ড, একটি অ্যাপ	টেস্টের নামগুলো — নামেই দাবি পড়া যায়
৩–৫	একটি টেস্ট ইচ্ছাকৃত ভাঙুন (`assertEqual` → `assertNotEqual`)	লাল আউটপুট, ব্যর্থতার বার্তা, কোন লাইনে
৫–৬	সেই টেস্টে `print()` বা `--pdb`	ডিবাগার থেমে যাওয়া — `n` , `p variable`
৬–৮	`python smoke_test.py`	৫৫ পাতা × ৪ ভূমিকা — সব ২০০/৪০৩, ৫০০ নেই

প্রস্তুতি (ক্লাসের আগে): `bash run.sh --reset` একবার চালিয়ে seeded ডেটাবেস তৈরি রাখুন।

সতর্কতা: `smoke_test.py` dev সেটিংসে চলে — PostgreSQL চালু না থাকলে প্রথমেই সংযোগ-ত্রুটি দেখাবে।

শিক্ষকের নোট: লাল আউটপুট দেখানোই সবচেয়ে জরুরি — শিক্ষার্থীরা প্রথমবার দেখে ভয় পায়, তাই ভাঙা-ঠিক করা একসাথে করুন।

ধাপ ১৬

নিজের প্রকল্পের টেস্ট-অডিট — ৮টি প্রশ্ন

প্রতিটির উত্তর "না" মানে একটি কাজ

প্রশ্ন	যদি উত্তর "না" হয়
১. পুরো স্যুট ২ মিনিটে চলে?	ধীর টেস্ট ভাগ করুন, `setUpTestData` ব্যবহার করুন
২. একটি টেস্ট ইচ্ছাকৃত ভেঙে লাল দেখেছেন?	প্রথমে সেটাই করুন — না হলে সবুজের ওপর ভরসা নেই
৩. প্রতিটি ভূমিকার জন্য একটি ৪০৩ টেস্ট আছে?	ভূমিকা × URL ম্যাট্রিক্স লিখুন
৪. ক্রস-ওয়ার্কস্পেস ৪০৪ টেস্ট আছে?	দ্বিতীয় প্রতিষ্ঠান বানিয়ে একটি টেস্ট
৫. নেগেটিভ টেস্টে প্রতিপক্ষ সারি নিজে বানানো?	seed-এর ওপর ভরসা বাদ দিন
৬. কোনো skip জমে আছে?	ঠিক করুন বা মুছুন — skip মানে অদৃশ্য ঋণ
৭. নাম দেখেই দাবি বোঝা যায়?	`test_<what>_<when>_<expected>`
৮. সবুজ স্যুটের পরেও `check --deploy` চালিয়েছেন?	টেস্ট আর ডিপ্লয়-চেক আলাদা কাজ করে

শিক্ষকের নোট: এই ছকটি হোমওয়ার্ক হিসেবে দিন — নিজের প্রজেক্টের আটটি উত্তর লিখে আনলে পরের ক্লাসে সরাসরি আলোচনা করা যাবে।

পর্ব ৫ সমাপ্তি

চার ক্লাসে যা শেখা হলো

৭০ স্লাইড · ১০৩ টেস্ট · ৭টি ফাঁক · ৩টি মিউটেশন মহড়া

ক্লাস	মূল ধারণা	দখল
২৪ · টেস্টের ভিত্তি	Arrange–Act–Assert · `setUpTestData` · `subTest`	একটি টেস্ট লেখা
২৫ · অনুমতি ও স্কোপ	`scoping.py` · ৪০৩/৪০৪ · `smoke_test`	কে কী দেখবে
২৬ · কভারেজ ও ফাঁক	শাখা কভারেজ · পিরামিড · ফ্লেকি · CI	কতটা আস্থা রাখা যায়
২৭ · ডিজাইন ও TDD	লাল-সবুজ-রিফ্যাক্টর · সমতুল্য শ্রেণি	কী টেস্ট করবেন, কী নয়

চিহ্নিত ফাঁক: reports ০ · audit ০ · আপলোড ০ · ক্রস-ওয়ার্কস্পেস কেবল students-এ · CI নেই · কভারেজ মাপা হয়নি · helper টেস্ট ≠ HTTP টেস্ট

পরের পর্বে (ডিপ্লয়) এই ফাঁকগুলোই **ঝুঁকি** হয়ে ফিরে আসে — কারণ সার্ভারে ভুল হলে টেস্ট নয়, ব্যবহারকারী প্রথম জানেন।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টে এই সাতটি ফাঁকের কোনটি আছে তা লিখতে — পরের ক্লাসে সেটাই আলোচনা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

এই পর্বের পুরো সারাংশ

নীতি	প্রমাণ
লাল না দেখলে সবুজের দাম নেই	TDD চক্র · ৩ ধাপ
শ্রেণি ভাবুন, সংখ্যা নয়	৩২ · ৩৩ · ৩৪ · ২৯৯ · ৩০০ · ৩০১
নেগেটিভ টেস্ট আগে	`404 foreign slug` · `403 before publish`
আপনার সিদ্ধান্ত টেস্ট করুন	লেবেল/HTML নয় — আচরণ
রিফ্যাক্টরের একমাত্র অনুমোদন: সবুজ	১০৩ টেস্ট · কোনো টেস্ট বদল নয়
ফাঁক লিখে রাখুন	`BUILD_STATUS.md` — "not faked, not skipped"

শিক্ষকের নোট: শেষ সারিটি আবার বলুন: "সৎ খাতা রাখা নিজের প্রতি সম্মান" — যা হয়নি তা লেখাও পেশাদারির অংশ।

অনুশীলন ও আলোচনা

২০ মিনিট — এক চক্র সম্পূর্ণ করুন

লাল থেকে সবুজ পর্যন্ত, এক বসায়

১. দাবি লিখুন (৩ মিনিট)

একটি বাক্যে: "...হলে ...হবে না" — সমতুল্য শ্রেণির ৩টি মান লিখুন।

২. লাল (৫ মিনিট)

টেস্ট লিখে চালান — ব্যর্থতার কারণটি কি প্রত্যাশিত?

৩. সবুজ (৭ মিনিট)

সবচেয়ে সহজ কোড; বাড়তি কিছু নয়।

৪. রিফ্যাক্টর (৫ মিনিট)

নাম পরিষ্কার করুন — টেস্ট বদলানো চলবে না।

- TDD ধীর করে না — কেন?
- কোন টেস্টটি আপনি লিখবেন না?
- রিফ্যাক্টরে টেস্ট বদলাতে হলে?
- প্রমাণ-খাতা কে পড়ে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সঠিক উত্তর: "টেস্ট বদলাতে হলে আচরণ বদলেছে — তাই সেটি রিফ্যাক্টর নয়, নতুন ফিচার।" আলাদা কমিট করুন।

২৮

পর্ব ৬ শুরু — "ল্যাপটপ থেকে সার্ভার পর্যন্ত"

ডিপ্লয়মেন্ট: `prod.py` · gunicorn · nginx · `collectstatic` · মাইগ্রেশন · `check --deploy` · ব্যাকআপ